messages of the specified type. This is shown in listing C.5. This is why you test the `chained` value in the destructor before waiting for messages; not only does it prevent moved-from objects waiting for messages, but it also allows you to transfer the responsibility of waiting to your new `TemplateDispatcher` instance.

Listing C.5. The TemplateDispatcher class template

```
namespace messaging
{
    template<typename PreviousDispatcher,typename Msg,typename Func>
    class TemplateDispatcher
    {
        queue* q;
        PreviousDispatcher* prev;
        Func f;
        bool chained;

        TemplateDispatcher(TemplateDispatcher const&)=delete;
        TemplateDispatcher& operator=(TemplateDispatcher const&)=delete;

        template<typename Dispatcher,typename OtherMsg,typename OtherFunc>
        friend class TemplateDispatcher;

        void wait_and_dispatch()
        {
            for(;;)
            {
                auto msg=q->wait_and_pop();
                if(dispatch(msg))
                    break;
            }
        }

        bool dispatch(std::shared_ptr<message_base> const& msg)
        {
            if(wrapped_message<Msg>* wrapper=
                dynamic_cast<wrapped_message<Msg>*>(msg.get()))
            {
                f(wrapper->contents);
                return true;
            }
            else
            {
                return prev->dispatch(msg);
            }
```

TemplateDispatcher instantiations are friends of each other

1 If we handle the message, break out of the loop

Check the message type, and call the function 2

3 Chain to the previous dispatcher